Calcul Scientifique

SMP5

Rachid Moustabchir
Departement de Physique

2025/10/01

Divers

- **Volume horaire : 12 × 1h45 cours, 9 × 1h45 TDs, 3 × 3h TPs.**
- **Prérequis du cours :**
  - Physique Générale :
    - Cours communs de Physique – S1, S2, S3, S4
  - Mathématiques :
    - Analyse – S1 et S3
    - Algèbre – S1 et S2
- **Informatique :**
  - Algorithmique et Programmation (Python) – S3
  - Langage de Programmation (TP) – Python
- **Note finale :** $(3EF + CT)/4$.
  - EF : Examen Final,
  - CT : Colle de TP,
- **mail: r.moustabchir@uiz.ac.ma**
- **web : http://ecours-fs.uiz.ac.ma/**
  - rubrique: Courses/Licence/SMP et PC-Physique/SMP5/ Calcul Scientifique

Objectifs

- Algorithmique numérique.
- Poursuite de l'apprentissage de Python.
- Analyser les points d'équilibre et la stabilité des systèmes dynamiques linéaires et non linéaires.
- Appliquer les outils de stabilité, notamment les théorèmes de Liapounov, pour identifier le comportement global et les attracteurs.
- Résolution numériques d'équations différentielles ordinaires (EDO) et les équations aux dérivées partielles (EDP).

Plan du cours

- **Chapitre I: Python pour la Simulation et la Visualisation**
- Chapitre II: Équations Différentielles Ordinaires (ODE)
- Chapitre III: Notions sur les systèmes dynamiques
- Chapitre IV: Résolution numérique des E.D.P.

Partie I:
Introduction à l'Algorithmique Numérique

Introduction

Définition Algorithmique : Petit Robert 97

- Enchaînement d'actions nécessaires à l'accomplissement d'une tâche.
  - Exemple: Donner un algorithme permettant de résoudre l'equation: ax = b.

Caractéristiques d'un bon algorithme:

- Il ne souffre d'aucune ambiguïté ⇒ très clair.
- Combinaison d'opérations (actions) élémentaires.
- Pour toutes les données d'entrée, l'algorithme doit fournir un résultat en un nombre fini d'opérations.

Première approche méthodologique:

1. Définir clairement le problème.
2. Rechercher une méthode de résolution (formules, ...).
3. Rédiger l'algorithme, en utilisant si nécessaire une méthode de raffinement progressif pour les cas complexes.

Pseudo-langage algorithmique

Vocabulaire de base:

- constantes, variables,
- opérateurs (arithmétiques, relationnels, logiques),
- expressions,
- instructions (simples et composées),
- fonctions.

Constantes, variables:

- Constante : symbole ou nom dont la valeur ne change pas.
- Variable : nom associé à une valeur modifiable, avec un type (entier, réel, caractère, etc.).
- Donnée : information fournie par l'utilisateur, souvent enregistrée dans une variable.

Pseudo-langage algorithmique

Opérateurs:

Opérateurs arithmétiques

Nom	Symbole	Exemple
Addition	+	a + b
Soustraction	-	a - b
Opposé	-	-a
Produit	*	a * b
Division	/	a / b

Opérateurs logiques

Nom	Symbole	Exemple
Négation	~	~a
OU	|	a|b
ET	&	a & b

Opérateurs relationnels

Nom	Symbole	Exemple
Égal à	==	a == b
Différent de	~=	a ~= b
Inférieur à	<	a < b
Supérieur à	>	a > b
Inférieur ou égal à	<=	a <= b
Supérieur ou égal à	>=	a >= b

Opérateur d'affectation

Nom	Symbole	Exemple
Affectation	=	a = b

Pseudo-langage algorithmique

Expression

- Une expression est un groupe de valeurs (comme des nombres, constantes ou variables) liées par des opérateurs (comme +, -, *, /) pour former un calcul qui donne une valeur.

Exemple d'expression numérique:

- $(b * b - 4 * a * c)/(2 * a)$

Opérandes: identifiants a, b, c, constantes 4 et 2.
Opérateurs: symboles *, - et /

Exemple d'expression booléenne:

- $(x < 3.14)$

Opérandes: identifiants x et contante 3.14
Opérateurs: symboles <

Pseudo-langage algorithmique

Instruction:

- Une instruction est un ordre donné à l'ordinateur pour exécuter une action.
- Il existe deux types d'instructions :
  - **Simple :** une seule action
    - Affectation d'une valeur à une variable. Exemple : x = 10
    - Appel d'une fonction ou procédure . Exemple : afficher(x) ou sqrt(a)
  - **Structurée :** ensemble d'instructions organisées (ex. : conditions, boucles...)
    - **Instructions composées :**Plusieurs instructions simples regroupées.
      - Exemple : un bloc d'instructions entre accolades { ... } ou begin ... end.
    - **Instructions répétitives (boucles) :**Répètent une ou plusieurs actions.
      - Exemples : pour, tant que, répéter...jusqu'à
    - **Instructions conditionnelles :**Ne s'exécutent que si une condition est vraie.
      - Exemple : si (a > 0) alors ...

Pseudo-langage algorithmique : Exemple : boucle «pour»

Algorithme Exemple boucle «pour»

Données : n, un entier positif

1: $S \leftarrow 0$
2: **Pour** $i \leftarrow 1$ à n faire
3: $S \leftarrow S + \cos(i\text{^}2)$
4: **Fin**

Mais que fait-il?
S=?

Listing: (Python) Example boucle for

```
import math  # Import du module math pour
utiliser la fonction cos()

# Demande à l'utilisateur de saisir une valeur
pour n
n = int(input("n = "))

# Vérifie que n est supérieur ou égal à 0
assert n >= 0, "n doit être supérieur ou égal à
0"

S = 0  # Initialisation de la somme

# Boucle de 1 à n (inclus)
for i in range(1, n + 1):
    S += math.cos(i ** 2)  # Ajout du cos(i^2)
à la somme

# Affichage du résultat
print("S =", S)
```

Pseudo-langage algorithmique : Exemple : boucle «tant que»

Algorithme Exemple boucle «tant que»

```
1: i←0, x←1
2: Tantque i<1000 faire
3:     x←x+i*i
4:     i←i+1
5: Fin
```

Listing: (Python) Example boucle while

```
i = 0
x = i

while i < 1000:
    x += i ** 2
    i += 1

print("x =", x)
```

Mais que fait-il?
i=? x=?

Pseudo-langage algorithmique : Exemple : instructions conditionnelles «si»

Algorithme Exemple instruction «si»

Données : age, un entier.

```
1: Si age >= 64 alors
2:     affiche('retraite')
3: Sinon Si age >= 18 alors
4:  affiche('majeur')
5: Sinon
6:  affiche('mineur')
7: Fin
```

Listing: (Python) Example instruction if

```
age = int(input("age = "))

if age >= 64:
    print("retraite")
elif age >= 18:
    print("majeur")
else:
    print("mineur")
```

Pseudo-langage algorithmique : méthodologie de construction

Description du problème:

- Définir les données disponibles
  - Issues de l'énoncé, des hypothèses ou de sources externes.
- Définir les objectifs à atteindre
  - Résultats attendus ou traitements à effectuer.
- Identifier les contraintes
  - Limites imposées par le contexte (temps, mémoire, précision...).

Recherche d'une méthode de résolution:

- Simplifier et clarifier le problème.
- Vérifier que le problème est bien solvable (pas d'indécidabilité).
- Chercher une stratégie de construction de l'algorithme.
- Décomposer le problème en sous-problèmes plus simples.
- Appliquer un raffinement successif, jusqu'à obtenir des instructions de base.

Écriture de l'algorithme:

- Spécifier les types des données et des résultats.
- L'algorithme doit produire au moins un résultat (numérique, graphique, etc.).
- Il doit s'exécuter en un nombre fini d'étapes.
- Sa description doit être claire et sans ambiguïté.

Pseudo-langage algorithmique : méthodologie de construction

Exercices:

Exercice 1

- Ecrire un algorithme permettant de calculer

$$S(x) = \sum_{k=1}^{n} k \sin(2 * k * x)$$

Exercice 2

- Ecrire un algorithme permettant de calculer

$$P(z) = \prod_{n=1}^{k} \sin(2 * k * z/n)^k$$

- Sa description doit être claire et sans ambiguïté.

Pseudo-langage algorithmique : méthodologie de construction

Correction:

Exercice 1

- Ecrire un algorithme permettant de calculer

$$S(x) = \sum_{k=1}^{n} k \sin(2 * k * x)$$

L'énoncé de cet exercice est imprécis. On choisi alors $x \in \mathbb{R}$ et $n \in \mathbb{N}$ pour rendre possible le calcul.

Algorithme Calcul de $S(x) = \sum_{k=1}^{n} k \sin(2 * k * x)$

Données : x : nombre réel,
n : nombre entier.
Résultat : S : un réel.
1: S ← 0
2: Pour k ← 1 à n faire
3: $S \leftarrow S + k * \sin(2 * k * x)$
4: Fin
5: Afficher S

Listing: (Python) Calcul de $S(x)$

```
import math

x = float(input("Entrez x : "))
n = int(input("Entrez n : "))

S = 0
for k in range(1, n + 1):
    S += k * math.sin(2 * k * x)

print("S =", S)
```

Pseudo-langage algorithmique : méthodologie de construction

Correction:

Exercice 2

- Ecrire un algorithme permettant de calculer

$$P(z) = \prod_{n=1}^{k} \sin(2 * k * z/n)^k$$

L'énoncé de cet exercice est imprécis. On choisi alors $z \in \mathbb{R}$ et $k \in \mathbb{N}$ pour rendre possible le calcul.

Algorithme Calcul de $P(z) = \prod_{n=1}^{k} \sin(2 * k * z/n)^k$

Données : z : nombre réel,
k : nombre entier.
Résultat : P : un réel.
1: P ← 1
2: Pour n ← 1 à k faire
3: $P \leftarrow P * \sin(2 * k * z/n)$
4: Fin
5: Afficher P

Listing: (Python) Calcul de $P(z)$

```
import math

k = int(input("Entrez la valeur de k : "))
z = float(input("Entrez la valeur de z : "))

P = 1
for n in range(1, k + 1):
    P *= math.sin((2 * k * z) / n) ** k

print("P =", P)
```

Pseudo-langage algorithmique : Les fonctions

Rôles des fonctions

- Les fonctions permettent :
  - d'automatiser des tâches répétitives dans un algorithme,
  - d'améliorer la clarté et la structure du code,
  - de réutiliser des portions de code dans d'autres algorithmes,
  - de modulariser la résolution d'un problème complexe …

Fonctions prédéfinies

- On distingue plusieurs catégories de fonctions souvent intégrées aux langages :
  - **Fonctions d'entrée/sortie :** Affiche, Lit, input(), print()...
  - **Fonctions mathématiques :** sin, cos, exp, sqrt, log, etc.
  - **Fonctions de gestion de fichiers :** open(), read(), write(), close()...
  - **Fonctions graphiques :** Tracé de courbes, affichage de figures, etc. (matplotlib, plot()...)

Pseudo-langage algorithmique : Les fonctions

Ecrire ses propres fonctions

- Avant d'écrire une fonction, il faut se poser les questions suivantes :
  - Quel est son objectif ?
    - Que doit-elle calculer ou réaliser précisément ?
  - Quelles sont ses données d'entrée ?
    - Identifier les paramètres nécessaires et leurs limitations (type, domaine de validité, etc.).
- Syntaxe: m arguments en entrée (données), 1 argument en sortie (résultat)

  Function $args \leftarrow \text{NomFonction}\ (\text{arg}e_1, \ldots, \text{arg}e_m)$
  Instructions
  Fin
- Syntaxe: m arguments en entrée (données), n arguments en sortie (résultats)

  Function $[\text{args}_1, \ldots, \text{args}_m] \leftarrow \text{NomFonction}\ (\text{arg}e_1, \ldots, \text{arg}e_m)$
  Instructions
  Fin

Pseudo-langage algorithmique : Les fonctions

Ecrire ses propres fonctions

- Ecrire une fonction permettant de résoudre:

$$ax = b$$

- But
  - Enoncé insuffisament précis! On choisi ici le problème:
    $a \in \mathbb{R}^*$ et $b \in \mathbb{R}$ donnés, trouver $x \in \mathbb{R}$ solution de $ax = b$
    On aurait pu prendre $\boldsymbol{a}$ une matrice réelle d'ordre n ($a \in \mathbb{R}^{n*}$) et $b \in \mathbb{R}^n$.
- Données :
  - $a \in \mathbb{R}^*$ et $b \in \mathbb{R}$.
- Résultats :
  - $x \in \mathbb{R}$.

Pseudo-langage algorithmique : Les fonctions

Ecrire ses propres fonctions

- Ecrire une fonction permettant de résoudre:

$$ax = b$$

REPD est le nom de la fonction

Algorithme Exemple de fonction : Résolution de l'équation du premier degré $ax = b$.

Données : a : nombre réel différent de 0,
b : nombre réel.
Résultat : x : un réel.
1: $x \leftarrow \text{REPD}(a, b)$
2: $x \leftarrow b/a$
3: Fin
4: Afficher x

Listing: (Python) Résolution de l'équation du premier degré $ax = b$.

```
def REPD(a, b):
    if a == 0:
        print("Erreur : a ne doit pas être nul")
        return None
    return b / a

# Programme principal
a = float(input("Entrez a : "))
b = float(input("Entrez b : "))

x = REPD(a, b)
print("x =", x)
```

Partie II:

Introduction à Python et Visualisation des Données

Visualisation des données

Types de graphiques courants :

- Courbes (line plots)
- Nuages de points (scatter plots)
- Courbes de niveau / cartes de densité (pour données en 2D)

Tracer les données:

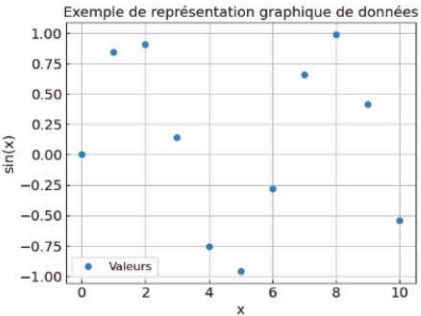
- Les programmes informatiques produisent des données numériques.
- Les nombres seuls ne permettent pas toujours de comprendre facilement le comportement d'un système ou ses propriétés.

Exemple:

- Considérons la fonction suivante : $y=\sin(x)$
- Calculons-la pour 10 points équidistants dans l'intervalle : $x=0,\ldots,10$

x (rad)	0	1	2	3	4	5	6	7	8	9	10
sin(x)	0	0.8415	0.909	0.141	-0.757	-0.959	-0.279	0.657	0.989	0.412	-0.544

Tracer les données : mise en graphique



Listing: (Python) Représentation graphique de données.

```
# Préliminaires : importer matplotlib et définir les styles par défaut
import matplotlib.pyplot as plt

# Paramètres de style par défaut (vous pouvez les modifier selon vos préférences)
params = {
    'legend.fontsize': 'large',      # Taille de la police de la légende
    'axes.labelsize': 'x-large',     # Taille des étiquettes des axes
    'axes.titlesize': 'x-large',     # Taille du titre
    'xtick.labelsize': 'x-large',    # Taille des graduations x
    'ytick.labelsize': 'x-large',    # Taille des graduations y
    'xtick.direction': 'in',         # Direction des graduations x
    'ytick.direction': 'in'          # Direction des graduations y
}
plt.rcParams.update(params)
```

```
# Définir les valeurs de x allant de 0 à 10
x = list(range(11))  # x de 0 à 10

# Valeurs correspondantes de sin(x), supposées en radians
y = [0.0000000, 0.8414709, 0.9092974, 0.1411200, -0.7568025,
     -0.9589243, -0.2794155, 0.6569866, 0.9893582, 0.4121185, -0.5440211]

# Tracer uniquement des points, sans ligne
plt.plot(x, y, marker='o', linestyle='None', label='Valeurs')

# Ajouter un titre au graphique
plt.title("Exemple de représentation graphique de données")

# Ajouter un label à l'axe des x
plt.xlabel("x")

# Ajouter un label à l'axe des y
plt.ylabel("sin(x)")

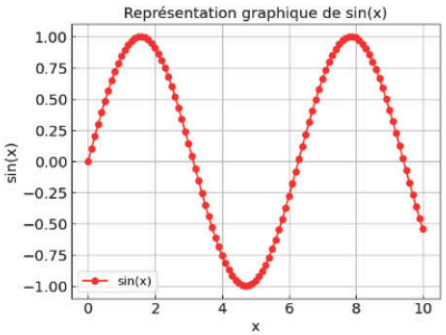
# Activer la grille
plt.grid(True)

# Afficher la légende
plt.legend()

# Sauvegarder le graphique dans un fichier PNG  sans couper les bords
plt.savefig("observation.png",  bbox_inches='tight')

# Afficher le graphique
plt.show()
```

Tracer les données : mise en graphique



Ajoutons davantage de points...

Nous avons maintenant suffisamment de points pour relier les valeurs par une courbe lisse.

Listing: (Python) Représentation graphique de données.

```
import matplotlib.pyplot as plt
import numpy as np  # Importation de NumPy avec l'alias standard 'np'

# x de 0 à 10 avec un pas de 0.1
x = np.arange(0, 10.1, 0.1)

# y = sin(x) en radians
y = np.sin(x)

# Tracer une ligne rouge avec des points
plt.plot(x, y, color='red', marker='o', linestyle='-', label='sin(x)')

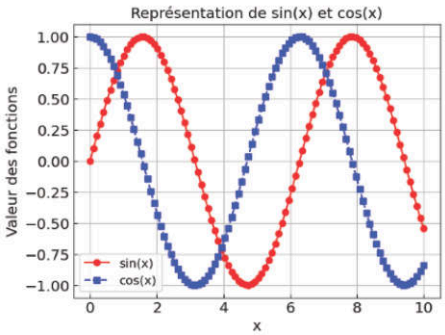
# Titre et étiquettes
plt.title("Représentation graphique de sin(x)")
plt.xlabel("x")
plt.ylabel("sin(x)")

# Grille et légende
plt.grid(True)
plt.legend()

# Sauvegarde sans couper les bords
plt.savefig("sinx_ligne_rouge.png", bbox_inches='tight')

# Affichage
plt.show()
```

Tracer les données : Tracer plusieurs courbes



Permet de comparer des fonctions, profils, ou des résultats expérimentaux entre eux.

Listing: (Python) Représentation graphique de données.

```
import matplotlib.pyplot as plt
import numpy as np

# Générations de 100 points entre 0 et 10
x = np.linspace(0, 10, 100)

# Calcul des valeurs sin(x) et cos(x)
ysin = np.sin(x)
ycos = np.cos(x)

# Tracer sin(x) : ligne rouge avec des cercles
plt.plot(x, ysin, color='red', marker='o', linestyle='-',
label='sin(x)')

# Tracer cos(x) : ligne bleue avec des carrés
plt.plot(x, ycos, color='blue', marker='s', linestyle='--',
label='cos(x)')

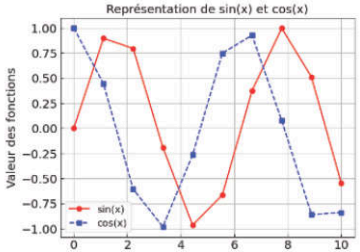
# Titre et étiquettes
plt.title("Représentation de sin(x) et cos(x)")
plt.xlabel("x")
plt.ylabel("Valeur des fonctions")

# Grille et légende
plt.grid(True)
plt.legend()

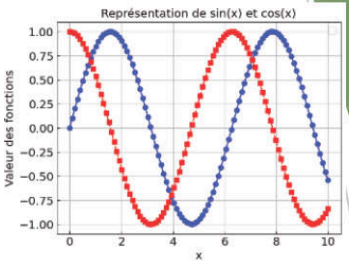
# Sauvegarde avec marges respectées
plt.savefig("sin_cos_plot.png", bbox_inches='tight')

# Affichage du graphique
plt.show()
```

Tracer les données : À éviter lors de la visualisation

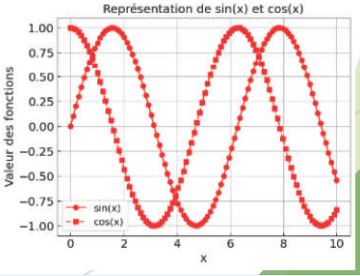


Trop peu de points (échantillonnage insuffisant)



Axes non étiquetés

Styles de courbes trop similaires (difficiles à distinguer)



Visualisation des données: Nuages de points (Scatter plots)

Types de graphiques courants :

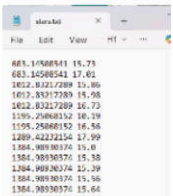
- Tous les points de données ne doivent pas être reliés par des lignes.
- Considérons par exemple des observations avec :
  - ✓ Température de surface d'une étoile (x)
  - ✓ Luminosité (y)
- Le nuage de points permet d'étudier :
  - ✓ La corrélation entre les variables
  - ✓ La structure globale des données
- Chaque point du nuage représente une paire (x, y).
- Très utile pour représenter des observations expérimentales.

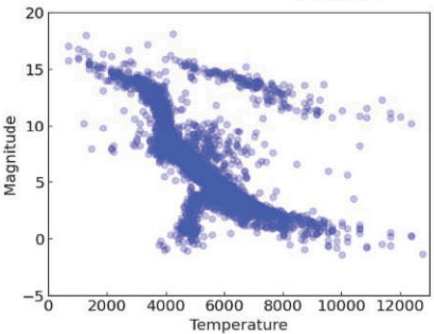
Exemple :

- Nuage de points représentant la distribution des températures de surface et de la luminosité des étoiles.
- from Mark Newman's book http://www-personal.umich.edu/~mejn/cp/chapters/graphics.pdf

Visualisation des données: Nuages de points (Scatter plots)

Distribution des températures de surface et de la luminosité des étoiles:





Listing: (Python) Représentation graphique de données.

```
import matplotlib.pyplot as plt
from numpy import loadtxt

# Charger les données de luminosité et température de surface à partir du fichier
data = loadtxt("stars.txt", float)

# Afficher les données chargées
print(data)

# Extraire la première colonne (température)
x = data[:, 0]

# Extraire la deuxième colonne (magnitude)
y = data[:, 1]

# Afficher les valeurs de température et magnitude
print(x)
print(y)

# Créer un nuage de points avec les données
plt.scatter(x, y, color="blue", alpha=0.25)

# Ajouter les labels des axes
plt.xlabel("Température (K)")  # Axe x : Température en Kelvin
plt.ylabel("Magnitude")        # Axe y : Magnitude (échelle de luminosité)

# Définir les limites des axes
plt.xlim(0, 13000)  # Température de 0 à 13000 K
plt.ylim(-5, 20)    # Magnitude de -5 à 20

# Sauvegarder le graphique sans couper les bords
plt.savefig("ScatterPlot.png", bbox_inches='tight')

# Afficher le graphique
plt.show()
```

Visualisation des données: Graphiques de contour et de densité

- Utilisés pour visualiser des fonctions à deux variables, par exemple : $f(x,y)$

Exemple : potentiel électrique d'un dipôle

- Le potentiel électrique d'un dipôle (en unités sans dimensions) s'écrit :

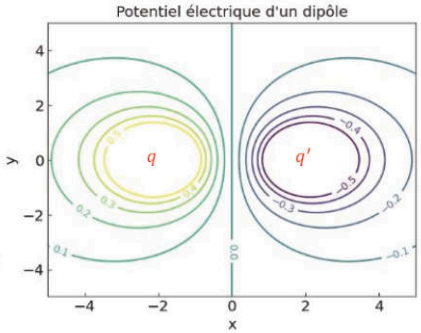
$$V_{dip}(x,y) = \frac{1}{\sqrt{(x-x_1)^2+(y-y_1)^2}} - \frac{1}{\sqrt{(x-x_2)^2+(y-y_2)^2}}$$

- où : (x_1, y_1) et (x_2, y_2) sont les coordonnées des deux pôles du dipôle.
- Ce type de champ apparaît par exemple dans :
  - Le potentiel électrique
  - Le champ gravitationnel
  - Les visualisations de champs vectoriels ou scalaires



Visualisation des données: Graphiques de contour et de densité

Tracer les lignes équipotentielles



Listing: (Python) Représentation graphique de données.

```
# Importer NumPy et Matplotlib
import numpy as np
import matplotlib.pyplot as plt

# Créer des grilles de valeurs x et y de -5 à 5 avec 200 points
x = np.linspace(-5, 5, 200)
y = np.linspace(-5, 5, 200)

# Définir les positions des deux charges du dipôle
x1, y1 = -2.0, 0.0  # Charge négative à (-2, 0)
x2, y2 = 2.0, 0.0   # Charge positive à (2, 0)

# Créer une grille 2D pour les calculs
X, Y = np.meshgrid(x, y)

# Calcul du potentiel électrique du dipôle (V = V1 - V2)
Vdip = 1.0 / np.sqrt((X - x1)**2 + (Y - y1)**2) - 1.0 / np.sqrt((X -
x2)**2 + (Y - y2)**2)

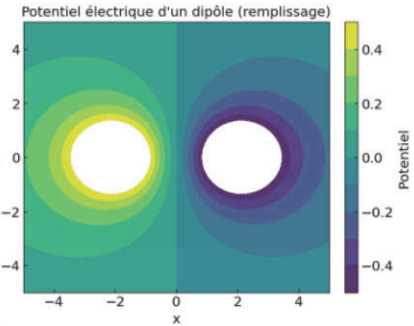
# Tracer les lignes équipotentielles
plt.title("Potentiel électrique d'un dipôle")
plt.xlabel("x")
plt.ylabel("y")
CS = plt.contour(X, Y, Vdip, levels=np.linspace(-0.5, 0.5, 11))  #
11 niveaux de -0.5 à 0.5
plt.clabel(CS)  # Ajouter des étiquettes aux lignes

# Sauvegarder le graphique
plt.savefig("Contour1.png", bbox_inches='tight')

# Afficher le graphique
plt.show()
```

Visualisation des données: Graphiques de contour et de densité

Tracer une version remplie (contourf) avec barre de couleur



Listing: (Python) Représentation graphique de données.

```
# Importer NumPy et Matplotlib
import numpy as np
import matplotlib.pyplot as plt

# Créer des grilles de valeurs x et y de -5 à 5 avec 200 points
x = np.linspace(-5, 5, 200)
y = np.linspace(-5, 5, 200)

# Définir les positions des deux charges du dipôle
x1, y1 = -2.0, 0.0  # Charge négative à (-2, 0)
x2, y2 = 2.0, 0.0   # Charge positive à (2, 0)

# Créer une grille 2D pour les calculs
X, Y = np.meshgrid(x, y)

# Calcul du potentiel électrique du dipôle (V = V1 - V2)
Vdip = 1.0 / np.sqrt((X - x1)**2 + (Y - y1)**2) - 1.0 / np.sqrt((X
- x2)**2 + (Y - y2)**2)

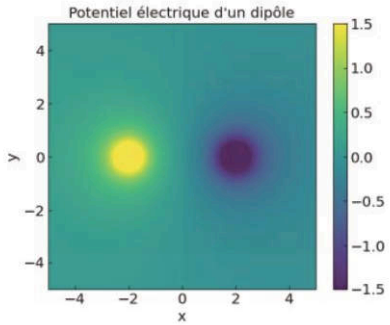
# Tracer une version remplie (contourf) avec barre de couleur
fig2, ax2 = plt.subplots(constrained_layout=True)
ax2.se:_title("Potentiel électrique d'un dipôle (remplissage)")
ax2.se:_xlabel("x")
ax2.se:_ylabel("y")
CS2 = ax2.contourf(X, Y, Vdip, levels=np.linspace(-0.5, 0.5,
11))  # Remplissage coloré
fig2.colorbar(CS2, label="Potentiel")  # Barre de couleur

# Sauvegarder le graphique sans couper les bords
plt.savefig("Contour2.png", bbox_inches='tight')

# Afficher le graphique
plt.show()
```

Visualisation des données: Graphiques de contour et de densité

distribution des températures de surface et de la luminosité des étoiles:



Listing: (Python) Représentation graphique de données.

```
# Importer NumPy et Matplotlib
import numpy as np
import matplotlib.pyplot as plt

# Créer des grilles de valeurs x et y de -5 à 5 avec 200 points
x = np.linspace(-5, 5, 200)
y = np.linspace(-5, 5, 200)

# Définir les positions des deux charges du dipôle
x1, y1 = -2.0, 0.0  # Charge négative à (-2, 0)
x2, y2 = 2.0, 0.0   # Charge positive à (2, 0)

# Créer une grille 2D pour les calculs
X, Y = np.meshgrid(x, y)

# Calcul du potentiel électrique du dipôle (V = V1 - V2)
Vdip = 1.0 / np.sqrt((X - x1)**2 + (Y - y1)**2) - 1.0 / np.sqrt((X -
x2)**2 + (Y - y2)**2)

# Autre exemple utilisant imshow (idéal pour une interpolation fluide
des couleurs)
plt.title("Potentiel électrique d'un dipôle")  # Titre du graphique
plt.xlabel("x")
plt.ylabel("y")
CS3 = plt.imshow(Vdip, vmax=1.5, vmin=-1.5,
origin="lower",extent=[-5,5,-5,5])
plt.colorbar(CS3)

# Sauvegarde du graphique sans rognage des bords
plt.savefig("Contour3.png", bbox_inches='tight')

# Affichage du graphique
plt.show()
```

Visualisation des données: Graphiques en 3D

- Ils permettent de représenter des relations mathématiques qui dépendent de deux variables indépendantes (x et y) et d'une variable dépendante (z=f(x,y)).

Pourquoi utiliser des tracés 3D ?

- **Visualisation intuitive :** Mieux comprendre la forme/comportement d'une fonction.
- **Analyse des phénomènes physiques :** Ondes, champs électromagnétiques...
- **Communication scientifique :** Illustration claire et esthétique pour les rapports...
- **Exploration de données :** Observation de variations dans l'espace (x,y,z).

Exemple :

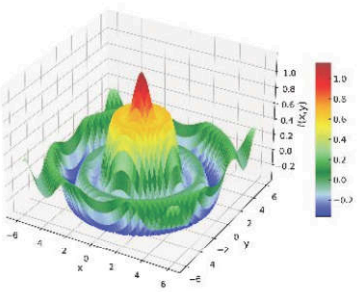
$$f(x,y) = \frac{\sin(\sqrt{x^2+y^2})}{\sqrt{x^2+y^2}} + 0.2\cos(3\sqrt{x^2+y^2}) = \frac{\sin(r)}{r} + 0.2\cos(3r)$$

- $r = \sqrt{x^2+y^2}$: distance radiale au centre.
- La fonction $\frac{\sin(r)}{r}$ crée des ondes concentriques (effet « goutte d'eau »).
- Le terme $0.2\cos(3r)$ ajoute des oscillations fines pour enrichir la surface.
- Résultat : une surface lisse et ondulée, parfaite pour illustrer les tracés 3D et la symétrie radiale.

Visualisation des données: Graphiques en 3D

Tracer les lignes équipotentielles

$$f(x,y) = \frac{\sin(\sqrt{x^2+y^2})}{\sqrt{x^2+y^2}} + 0.2\cos(3\sqrt{x^2+y^2})$$



Listing: (Python) Représentation graphique de données.

```
from matplotlib.colors import LinearSegmentedColormap
from mpl_toolkits.mplot3d import Axes3D

# ---------- Définition de la fonction ----------
def surface_function(x, y):
    r = np.sqrt(x**2 + y**2)
    z = np.sin(r) / r + 0.2 * np.cos(3 * r)
    return z

# ---------- Domaine ----------
x = np.linspace(-6, 6, 400)
y = np.linspace(-6, 6, 400)
X, Y = np.meshgrid(x, y)                    # Créer une grille 2D pour les calculs
Z = surface_function(X, Y)

# ---------- Définir colormap ----------
colors = ["blue", "lightblue", "green", "lightgreen", "yellow", "orange", "red", "brown"]
custom_cmap = LinearSegmentedColormap.from_list(
    "BlueLightBlueGreenLightGreenYellowOrangeRedBrown", colors
)

# ---------- Tracé 3D ----------
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

surf = ax.plot_surface(X, Y, Z, cmap=custom_cmap, edgecolor='none',
antialiased=True, alpha=0.95)

ax.set_title(r"$f(x,y) = \frac{\sin(\sqrt{x^2+y^2})}{\sqrt{x^2+y^2}} +
0.2\cos(3\sqrt{x^2+y^2})$")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("f(x,y)")

fig.colorbar(surf, shrink=0.5, aspect=10)

plt.tight_layout()
plt.savefig("Surface_3D_CustomColors.png", dpi=300, bbox_inches='tight')
plt.show()
```